

COMPUTER NETWORKS

Week-10

The Internet Transport Protocols

- TCP (Transmission Control Protocol)
- UDP (User Datagram Protocol)

TCP

- Internetworking protocol
- Resides within Layer 4 (Transport Layer) of the OSI model
- It provides an end-to-end transport of data units using connection oriented services across multiple packet switching networks
- Provides reliable host-to-host data transfer

TCP

- Connection Oriented protocol
- A connection need to be established (using 3-way handshaking) before data is transmitted using TCP
- TCP data transfer takes more time (3-way handshaking is done for establishing connection and then for removing the connection) than data transferred using UDP
- Full Duplex Mode

TCP

- Header Size of a TCP packet is bigger than the UDP header
- TCP does the error checking also
- Packets are ordered i.e they are received in the same order as they are sent
- Application layer protocols like HTTP, FTP, Telnet, etc. uses TCP

UDP

- UDP is Connectionless protocol and no connection needs to be established
- The packets are sent directly over the network
- Data transfer using TCP is reliable and UDP is non-reliable (sender does not know, for sure, if the packet has actually reached the receiver or not)
- no sender-receiver handshaking
- UDP segments may be:
 - lost
 - delivered out-of-order

UDP

- Does not have an option for Error checking
- Used by protocols like VoIP, DHCP, SNMP, RIP, DNS etc.
- Full Duplex
- Unreliable but FAST
- Applications must deal with lost packets

TCP vs UDP

- TCP
 - Connection oriented
 - Confirmed data transfers
 - High Overhead
 - Header of about 20 bytes
 - Flow Control
- UDP
 - Connectionless
 - Unconfirmed service
 - Low overhead
 - Header of 8 bytes No flow Control

DHCP

- Dynamic Host Configuration Protocol (**DHCP**) is a client/server protocol that automatically provides an Internet Protocol (IP) host with its IP address and other related configuration information such as the subnet mask and default gateway
- Used in :
 - Home Wi-Fi routers (automatically assign IPs to phones/laptops)
 - School or university labs
 - Office networks
 - Large enterprise network

SNMP

- Simple Network Management Protocol (**SNMP**) is a networking protocol used for the management and monitoring of network-connected devices in Internet Protocol networks
 - E.g.
 - A university network with 50 computers, 5 switches, and 2 routers
- The admin can use SNMP to:
- Monitor traffic on each switch
 - Get alerts when a router fails
 - Check which device is overloaded

VoIP

- Voice over Internet Protocol (**VoIP**), is a technology that allows you to make voice calls using a broadband Internet connection instead of a regular (or analog) phone line
- No need for separate telephone infrastructure
- Works over Lan/WiFi
- E.g.
- Used in MS-Teams, Zoom, customer support call centers, help desks, employees in an organization, Apps like WhatsApp, etc.
- Voip : Voice travels like data on internet (as highway)

Book Reference

- Chapter 5
- James Kurose, W. Ross

- Chapter 18
- Douglas E. Comer

Checksum

- A checksum is a small value calculated from a block of data
- Used to detect errors during transmission or storage
- It's like a **digital fingerprint** of the data

Checksum- Why?

- When data travels across a network, it can get corrupted due to:
- Noise
- Signal interference
- Hardware issues
- So we need a way to check whether the data received is exactly the same as what was sent

Checksum- How does it work?

- Sender side:
 - Take the data
 - Apply a simple mathematical operation (like addition)
 - Generate a checksum
 - Send data + checksum
- Receiver side:
 - Perform the same calculation on received data
 - Compare with received checksum
 - Matched → Data is correct
 - NOT matched → Error detected

Checksum-Ex-1

- Suppose data is: 10, 20, 30
- **Sender** computes:
- Checksum = $10 + 20 + 30 = 60$
- **Sender** sends:
- Data: 10, 20, 30
- Checksum: 60
- **Receiver** computes again:
- $10 + 20 + 30 = 60$
- No error
- But if data changes during transmission:
- 10, 25, 30 $\rightarrow$ sum = **65**
- Error detected
- **Checksum does not correct errors**
- **It only detects errors**

Error Checking

- Transport Layer
- This is where **checksum** is most commonly used.
- Protocols like TCP and UDP (on Transport layer) use checksums to detect errors in data
- The sender calculates a checksum and attaches it to the segment.
- The receiver recalculates it and compares:
 - Same → data is correct
 - Different → data is corrupted
- Example:
 - TCP checksum ensures reliable communication
 - UDP checksum provides optional error detection

Checksum

- A host can transmits a packet to the nearest router, either on its own LAN or over a point-to-point link to the ISP
- The packet is stored there until it has fully arrived and the link has finished its processing by verifying the checksum

Checksum

- Checksum is a simple error detection mechanism to determine the integrity of the data transmitted over a network
- Communication protocols like TCP/IP, UDP implement this scheme in order to determine whether the received data is corrupted along the transmission in a network

Checksum

- The sender of an IPv4 datagram would compute the checksum value based on the data and embed it in the frame
- The receiver would also compute the checksum locally and based on the result ascertain the data integrity

Let's start with a simple Ex-2

- Suppose we have a list of five 4-bit numbers (7, 11, 12, 0, 6) that we want to send to a destination
- So we will send the sum of the numbers along with the original numbers
- We send (7, 11, 12, 0, 6, 36), where 36 is the sum of the original numbers
- The receiver adds the five numbers and compares the result with the sum
- If the two are the same, the receiver assumes **no error**, accepts the five numbers, and discards the sum
- Otherwise → data **NOT accepted**

Let's work it out differently

- To make the job of the receiver easier
- Send the negative (complement) of the sum, called the checksum
- So we send (7, 11, 12, 0, 6, -36)
- The receiver adds all the numbers received (including the checksum)
- If the result is 0, it assumes no error
- otherwise, there is an **error**

Revisiting Ex-1

SENDER Side			
	7		
	11		
	12		
	0		
	6		
	0		
Sum	36		
Binary of 36	100100		
we have 4-bits			
wrapping bits	0100		
	+	10	
	0110	=6	
1'S Complement	1001		
checksum	1001	=9	
Data Sent	7 11 12 0 6 9		

RECEIVER Side			
	7		
	11		
	12		
	0		
	6		
	9		
Sum	45		
Binary of 45	101101		
we have 4-bits			
wrapping bits	1101		
	+	10	
	1111	=15	
1'S Complement	0000		
checksum	0000	=0	
Data Verified			

Sender Side

1. The message is divided into 4-bit values
2. The value of the checksum word is set to 0
3. All values including the checksum are added
4. The sum is complemented and becomes the checksum
5. The checksum is sent with the data

Revisiting Ex-1

SENDER Side			
	7		
	11		
	12		
	0		
	6		
	0		
Sum	36		
Binary of 36	100100		
we have 4-bits			
wrapping bits	0100		
	+	10	
	0110	=6	
1'S Complement	1001		
checksum	1001	=9	
Data Sent	7 11 12 0 6 9		

RECEIVER Side			
	7		
	11		
	12		
	0		
	6		
	9		
Sum	45		
Binary of 45	101101		
we have 4-bits			
wrapping bits	1101		
	+	10	
	1111	=15	
1'S Complement	0000		
checksum	0000	=0	
Data Verified			

Receiver Side

1. The message (including checksum) is divided into 4-bit values
2. All values are added
3. The sum is complemented and becomes the new checksum
4. If the value of **checksum is 0**, the message is accepted
5. Otherwise, it is rejected

Example-2

- Data in hexadecimal format to be sent
- 3456 ABCC 02BC EEEE
- Calculate the check sum

Add them up using calculator

		3	4	5	6
		A	B	C	C
		0	2	B	C
		E	E	E	E
Initial Cheksum		0	0	0	0

Addition done

carry				2	
		3	4	5	6
		A	B	C	C
		0	2	B	C
		E	E	E	E
Initial Cheksum		0	0	0	0
	Sum			C	C

Addition done

carry		2	2	2	
		3	4	5	6
		A	B	C	C
		0	2	B	C
		E	E	E	E
Initial Cheksum		0	0	0	0
	Sum		1	C	C

Addition done

carry	1	2	2	2	
		3	4	5	6
		A	B	C	C
		0	2	B	C
		E	E	E	E
Initial Cheksum		0	0	0	0
	Sum	D	1	C	C

Wrapping the carry

	SENDER Side				
carry	1	2	2	2	
		3	4	5	6
		A	B	C	C
		0	2	B	C
		E	E	E	E
Initial Cheksum		0	0	0	0
	Sum	D	1	C	C
wrapping carry					1
	SUM	D	1	C	D

Taking 1's Complement

1'S Complement				
		D =	1101	
			0010	=2
		1 =	0001	
			1110	=E
		C =	1100	
			0011	=3
		D =	1101	
			0010	=2

Checksum Finally..

	SENDER Side				
carry	1	2	2	2	
		3	4	5	6
		A	B	C	C
		0	2	B	C
		E	E	E	E
Initial Cheksum		0	0	0	0
	Sum	D	1	C	C
wrapping carry					1
	SUM	D	1	C	D
	checksum	2	E	3	2

Data to be sent..?

	SENDER Side				
carry	1	2	2	2	
		3	4	5	6
		A	B	C	C
		0	2	B	C
		E	E	E	E
Initial Cheksum		0	0	0	0
	Sum	D	1	C	C
wrapping carry					1
	SUM	D	1	C	D
	checksum	2	E	3	2
	Data Sent	3456	ABCC	02BC	EEEE 2E32

Verify at the Receiver side

	RECEIVER Side				
carry	1	2	2	2	
		3	4	5	6
		A	B	C	C
		0	2	B	C
		E	E	E	E
Initial Cheksum		2	E	3	2
	Sum	F	F	F	E

Verified..

	RECEIVER Side				
carry	1	2	2	2	
		3	4	5	6
		A	B	C	C
		0	2	B	C
		E	E	E	E
Initial Cheksum		2	E	3	2
Sum		F	F	F	E
wrapping carry					1
SUM		F	F	F	F
1'S Complement					
		F =	1111		
			0000	=0	
checksum	0	0	0	0	
Data Sent	3456	ABCC	02BC	EEEE	2E32
NO ERROR FOUND AT RECEIVER END					

Example-3

- Find the checksum at the receiver site/side if the following data is received (checksum in blue color)

3456 ABCE 02BC EEEE 2E32

Example - 4

- You are required to apply checksum as an error detection mechanism. You have to send the given data using 16-bit words:
- 4500 0034 B612 4000 4006 0A00 008B 5BC6
AEE0
- Show complete working at the sender site as well as receiver site and give your conclusion if correct data has been sent or not?

Ex-5

- You are using 16-bit 1's complement checksum technique to send the following data:
 - ABCD 1234 F0F0 0F0F AAAA 5555 8001 7FFE
1. Calculate the checksum for the original data
 2. Write the final transmitted data from the sender

Ex-5

- You are using 16-bit 1's complement checksum technique to send the following data:
 - ABCD 1234 F0F0 0F0F AAAA 5555 8001 7FFE
1. Calculate the checksum for the original data

Sender Side

	3	3	3	3	
		A	B	C	D
		1	2	3	4
		F	0	F	0
		0	F	0	F
		A	A	A	A
		5	5	5	5
		8	0	0	1
		7	F	F	E
Initial Checksum		0	0	0	0
			1	1	
Sum		B	D	F	E

Sender Side

	3	3	3	3	
		A	B	C	D
		1	2	3	4
		F	0	F	0
		0	F	0	F
		A	A	A	A
		5	5	5	5
		8	0	0	1
		7	F	F	E
Initial Checksum		0	0	0	0
			1	1	
Sum		B	D	F	E
Carry Wrap				+	3
Final Sum		B	E	0	1

Calculate the Checksum value?

Sender Side

	3	3	3	3	
		A	B	C	D
		1	2	3	4
		F	0	F	0
		0	F	0	F
		A	A	A	A
		5	5	5	5
		8	0	0	1
		7	F	F	E
Initial Checksum		0	0	0	0
			1	1	
Sum		B	D	F	E
Carry Wrap				+	3
Final Sum		B	E	0	1
		1011	1110	0000	0001
1'S Complement		0100	0001	1111	1110
HexaDec		4	1	F	E
Checksum		4	1	F	E

Ex-5

- You are using 16-bit 1's complement checksum technique to send the following data:
 - ABCD 1234 F0F0 0F0F AAAA 5555 8001 7FFE
1. Calculate the checksum for the original data
 2. Write the final transmitted data from the sender
 3. During transmission, the following were sent incorrectly: F0F0 → **F0F1** AND 0F0F → **0F0E**

Ex-5

- You are using 16-bit 1's complement checksum technique to send the following data:
- ABCD 1234 F0F0 0F0F AAAA 5555 8001 7FFE

Write the received data at the receiver end

ABCD 1234 F0F1 0F0E AAAA 5555 8001 7FFE
41FE (CHECKSUM)

Recalculate at the receiver end and give your conclusion

Ex-5

- Original Sum + checksum = ?

Ex-5

- Original Sum + checksum = ?
- $BE01 + 41FE = FFFF$
- New Sum with corrupted values ?

Ex-5

- Original Sum + checksum = ?
- $BE01 + 41FE = FFFF$
- New Sum with corrupted values ? $FFFC$ with 3 carry
- $FFFC + 3 = FFFF \rightarrow$ error goes undetected
- Limitation of 1's complement arithmetic
- Only verifies overall sum
- Checksum fails when multiple errors balance each other
 $\rightarrow$ producing same final sum

Think about..

- Where Checksum is used / which protocols use it and why?
- The above is included in your syllabus

Book-B

- Chapter 24,8
- Douglas E. Comer